

Audio Signal Analysis for Emotion

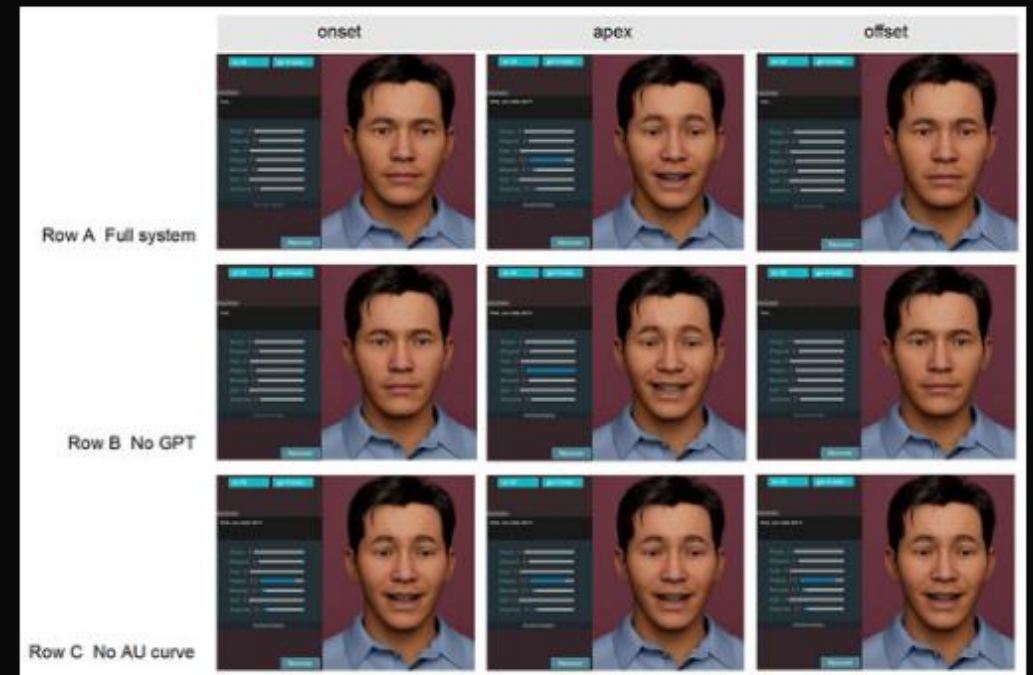
A Comprehensive Voice Monitoring & Emotion Detection
System

Under the supervision of:
**Dr. Doaa Adel | Eng. Fatma
Elzahraa**

Project Overview

This project bridges Hardware and Software to analyze human voice patterns. By capturing audio peaks via Arduino and processing them in real-time with Python, the system classifies the speaker's emotional state based on sound intensity.

- Real-time Signal Acquisition
- Digital Signal Processing (DSP)
- Emotion Classification Logic



| The Challenge



Latency

Detecting emotion in real-time requires high-speed data processing without delay.



Noise

Environmental noise can distort audio readings, requiring a baseline calculation.

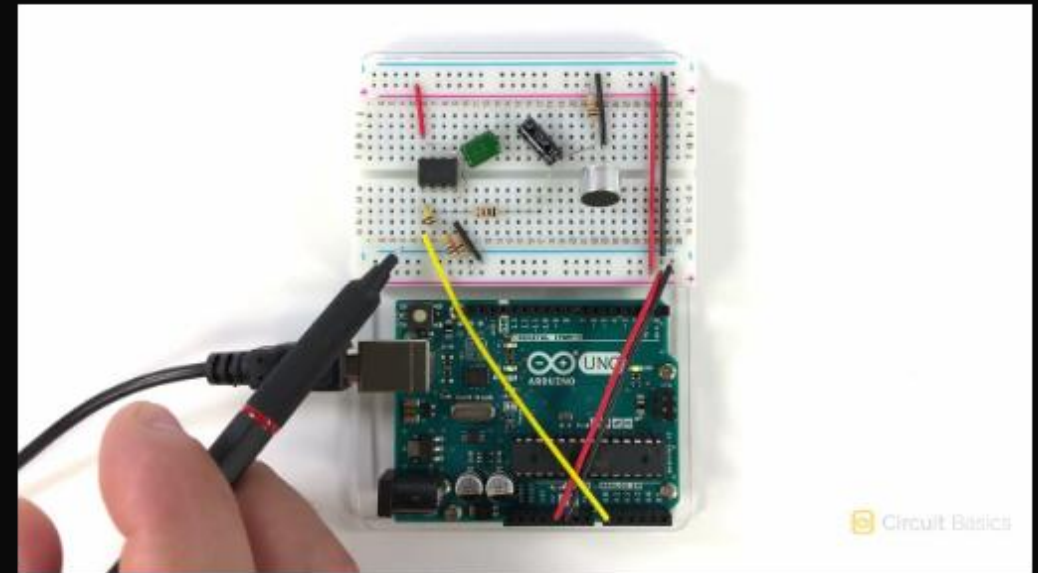


Visualization

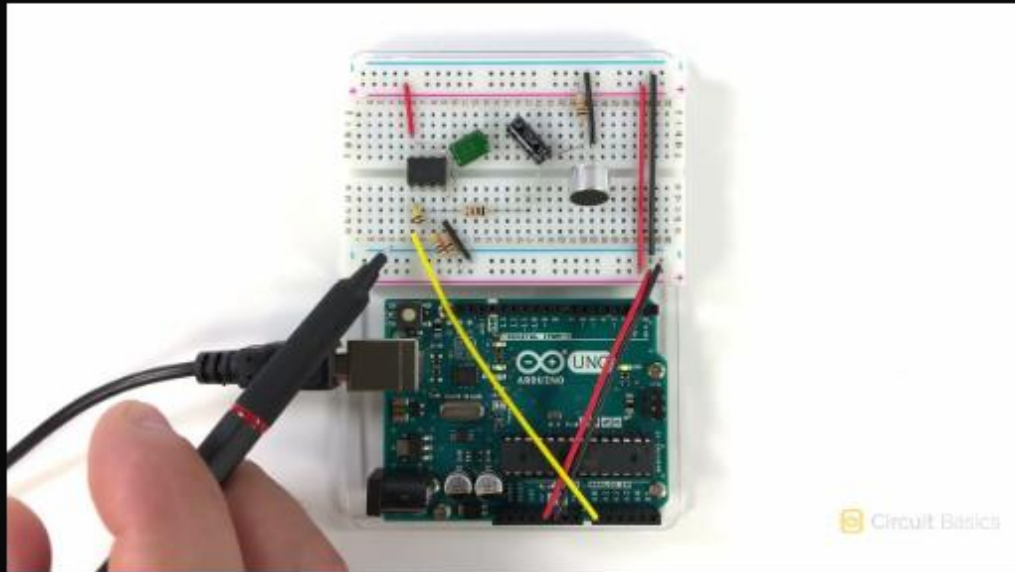
Representing complex waveforms in a way that is intuitive for users.

| System Architecture

- **Hardware Layer:** Arduino Uno + High-Sensitivity Microphone Sensor.
- **Transport Layer:** Serial Communication (USB) at 9600 Baud Rate.
- **Software Layer:** Python (Tkinter GUI, Matplotlib Graphics).
- **Analysis Layer:** Moving Average Smoothing & Baseline Adaptation.



Hardware Layer



Input Mechanism

The system utilizes the **Analog Input (A0)** of the Arduino. The microphone captures sound pressure waves and converts them into voltage fluctuations.

- Operating Voltage: 5V
- Sensitivity Adjustment: Potentiometer on Mic Module
- Sample Window: 50ms for stable signal detection

| Signal Logic

To accurately measure sound intensity, we use the **Peak-to-Peak** method within the Arduino loop. This filters out DC offset and focuses on wave magnitude.

$$V_{PP} = \text{Signal}_{\text{max}} - \text{Signal}_{\text{min}}$$

The resulting value represents the true loudness of the ambient sound, which is then transmitted via `Serial.println()`.

Python Environment

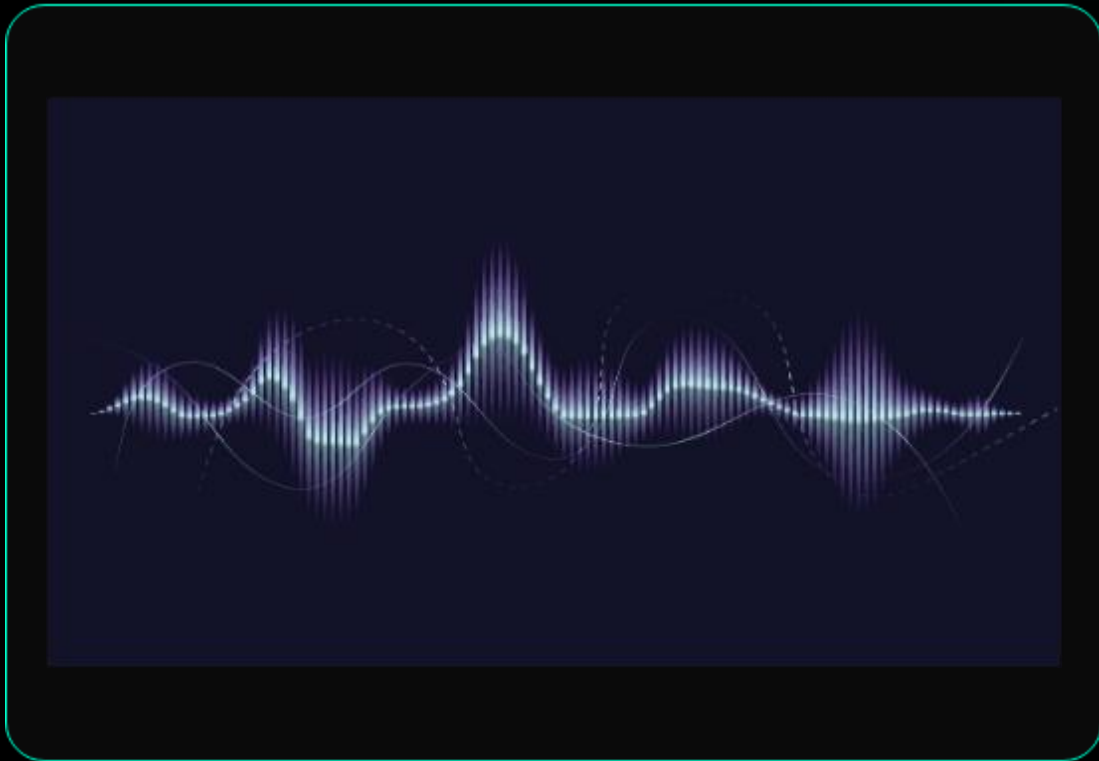
Libraries Used

- PySerial: Connects Python to Arduino hardware.
- Tkinter: Professional Dark-mode GUI development.
- Collections (Deque): Efficient real-time data buffering.
- NumPy: Mathematical calculations for baseline averages.

```
#!/usr/bin/env python3
from Crypto.PublicKey import RSA
from secret import FLAG

for i in range(1, 6):
    with open('{}pub.pem'.format(i), 'rb') as f:
        key = RSA.importKey(f.read())
    with open('{}enc'.format(i), 'wb') as f:
        f.write(key.encrypt(FLAG.encode(), None)[0])
```

| Live Visualization



Smooth Graph Engine

The UI integrates a **Matplotlib** figure into a Tkinter canvas. To prevent jittery lines, we implement a smoothing loop:

```
avg = sum(temp[start:i+1]) / len(temp[start:i+1])
```

This provides a professional, fluid "medical-grade" waveform display.

| Emotion Classification

The system adaptively learns the room's noise floor (Baseline) and categorizes input based on deviation thresholds:

Calm 😊

Deviation < 10 units above baseline.
Indicates silence or soft speaking.

Normal 😊

Deviation < 40 units. Represents
conversational speaking levels.

Angry 😡

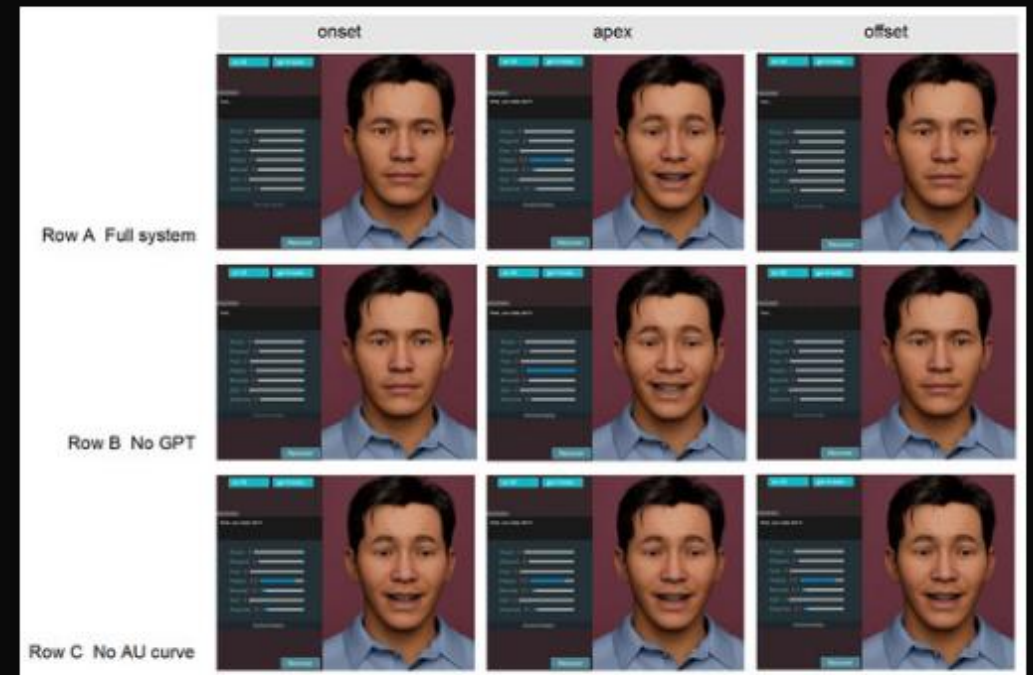
Deviation > 40 units. Sharp spikes
indicating shouting or high emotion.

Baseline Adaptation

How it learns?

The program maintains a baseline list of the first 50 readings. This allows the system to calibrate itself to different room environments before starting the analysis.

- Dynamic Calibration
- Buffer size: 15 points
- Moving Average: 20ms refresh rate



| Project Team

Name
Yousef Mohamed Elsayed
Khaled ELSAYED mohamed mohamed
Abdelrhman Saeed Tawakol
Ebrahim Abdelsalam gamh
Omar ahmed ahmed
Ahmed abdelaziz mohamed abbas

Thank You

We extend our deepest gratitude to our mentors for their guidance.

Dr. Doaa Adel | Eng. Fatma Elzahraa

